Exercise 4
JOINS, Unions, Filtering, and Aggregates.

Table 1: Employees.

EmployeeID	FirstName	LastName	Department	Salary	City	Age
1	John	Doe	IT	70000	New York	35
2	Alice	Smith	HR	60000	Los Angeles	40
3	Bob	Johnson	Finance	75000	Toronto	45
4	David	Brown	IT	72000	London	30
5	Emma	Wilson	Sales	65000	Sydney	28
6	Michael	Clark.	Finance	80000	New York	50

Table 2: Projects.

Project ID	EmployeeID	ProjectName	Budget	StartDate	Status
101	1	AI Development	100000	2024-01-10	Completed
102	2	EmployeeTraining	50000	2024-02-15	Ongoing
103	3	Cybersecurity Audit	75000	2024-03-05	Pending
104	4	Financial Analysis	90000	2024-04-12	Ongoing
105	5	Market Expansion	65000	2024-05-20	Complete
106	6	Risk Management	80000	2024-06-10	Pending

SQL Joins

Retrieve all employees and their assigned projects using an INNER JOIN.

Return: EmployeeID, first-name, LastName, Department, Salary, ProjectID, ProjectName, Budget, Status.

```
SELECT e.EmployeeID, FirstName, LastName, Department,
       salary, ProjectID, ProjectName, Budget, Status
FROM Employees AS e
INNER JOIN Projects AS p
ON e.Employee ID = p.EmployeeID;
```

EmployeeID	FirstName	LastName	Department	Salary	ProjectID	ProjectName	Budget	Status
1	John	Doe	IT	70000	101	AI Development	100000	Completed
1	John	Doe	IT	70000	103	Cybersecurity Audit	75000	Pending
2	Alice	Smith	HR	60000	102	Employee Training	50000	Ongoing
3	Bob	Johnson	Finance	75000	104	Financial Analysis	90000	Ongoing
5	Emma	Wilson	Sales	65000	105	Market Expansion	65000	Completed
6	Michael	Clark	Finance	80000	106	Risk Management	80000	Pending

Retrieve all employees and their assigment projects, including employees who have no projects using a LEFT JOIN

Return: EmployeeID, FirstName, LastName, Department, Salary, ProjectID, ProjectName, Budget, Status (LEFT)

```
SELECT e.EmployeeID, FirstName, LastName, Department,
       salary, ProjectID, ProjectName, Budget, Status.
FROM employees AS e
LEFT JOIN projects A p
ON e.EmployeeID = p.EmployeeID;
```

Retrieve all projects and their assigned employees, including projects that have no employees using a RIGHT JOIN.

Return: ProjectID, ProjectName, Budget, Status, EmployeeID, FirstName, LastName, Department, S

```
SELECT  projectID,
        ProjectName,
        Budget,
        Status,
      e.EmployeeID,
        FirstName,
        LastName,
        Department,
        Salary
FROM Employees AS e
RIGHT JOIN Projects AS p
ON e.EmployeeID = p.EmployeeID;
```

EmployeeID	FirstName	LastName	Department	Salary	ProjectID	ProjectName	Budget	Status
1	John	Doe	IT	70000	101	AI Development	100000	Complete
2	Alice	Smith	HR	60000	102	Employee Training	50000	Ongoing
3	Bob	Johnson	Finance	75000	104	Financial Analysis	90000	Ongoing
4	David	Brown	IT	72000	null	null	null	null
5	Emma	Wilson	Sales	65000	105	Market Expansion	65000	Complete
6	Michael	Clark	Finance	80000	106	Risk Management	80000	Pending
1	John	Doe	IT	70000	103	Cybersecurity Audit	75000	Pending

ProjectID	ProjectName	Budget	Status	EmployeeID	FirstName	LastName	Department	Salary
101	AI Development	100000	Completed	1	John	Doe	IT	70000
102	Employee Training	50000	Ongoing	2	Alice	Smith	HR	60000
103	Cybersecurity Audit	75000	Pending	1	John	Doe	IT	70000
104	Financial Analysis	90000	Ongoing	3	Bob	Johnson	Finance	75000
105	Market Expansion	65000	Completed	5	Emma	Wilson	Sales	65000
106	Risk Management	80000	Pending	6	Michael	Clark	Finance	80000

Retrieve all employees and projects including those without a match in either table using a FULL OUTER JOIN.

Return: EmployeeID, FirstName, LastName, Department, Salary
ProjectID, ProjectName, Budget, Status.

```
SECECT  p.EmployeeID,
        FIRSTName,
        LastName,
        Department,
        Salary,
        ProjectID,
        ProjectName,
        Budget,
        Status
FROM employees AS e
FULL OUTER JOIN AS p
ON e.EmployeeID = p.EmployeeID;
```

EmployeeID	FirstName	LastName	Department	Salary	ProjectID	Project Name	Budget	Status
1	John	Doe	IT	70000	AI Development	101	100000	Completed
2	Alice	Smith	HR	60000	Employee Training	102	50000	Ongoing
3	Bob	Johnson	Finance	75000	Finance Analysis	104	90000	Ongoing
4	Davia	Brown	IT	72000	null	null	null	null
5	Emma	Wilson	Sales	65000	Market Expansion	105	65000	Completed
6	Michael	Clark	Finance	80000	Risk Management	106	80000	Pending
1	John	Doe	IT	70000	Cybersecurity Audit	108	75000	Pending

FILTERING STATEMENTS

Retrieve employees who earn more than 70,000

```
SELECT   employeeID,
         FirstName,
         LastName,
         Department,
         Salary,
FROM employees
WHERE  salary > 70 000
```

Employee ID	FirstName	LastName	Department	Salary
3	Bob	Johson	Finance	75000
4	David	Brown	IT	72000
6	Michael	Clark	Finance	80000

Retrieve projects that have a budget greater than 70,000 and are not completed.

```
SELECT   ProjectID,
         ProjectName,
         Budget,
         Status
FROM  Projects
WHERE     budget > 70000
      AND  status = 'PENding'
       OR  status = 'ongoing'
```

Project ID	ProjectName	Budget	Status
103	Cybersecurity Audit	75000	Pending
104	Financial Analysis	90000	Ongoing
106	Risk Management	80000	Pending

Retrieve employees from New York OR Toronto, ordered by salary in descending order.

```
SELECT  EmployeeID,
        FirstName,
        Department,
        Salary,
        City
FROM  Employees
WHERE  City IN ('New York', 'Toronto')
ORDER BY salary DESC;
```

EmployeeID	FirstName	Department	Salary	City
6	Michael	Finance	80000	New York
3	Bob	Finance	75000	Toronto
1	John	IT	70000	New York.

Retrieve the top 3 highest paid employees.

```
SELECT  Employee ID,
        FirstName,
        LastName
        Department,
        Salary
FROM  employees
ORDER BY Salary DESC
LIMIT 3;
```

Employee ID	FirstName	LastName	Department	Salary
6	Michael	Clark	Finance	80000
3	Bob	Johnson	Finance	75000
4	David	Brown	IT	72000

Aggregate Functions with Groupby and having

Find the total salary per department, sorted in descending order.

```
SELECT   department,
         SUM(salary) AS TotalSalary
FROM employees
GROUP BY department,
ORDER BY TotalSalary DESC;
```

Department	TotalSalary
Finance	155000
IT	142000
Sales	65000
HR	60000

Find the average salary per city, but only include cities where the average salary is greater than 65000

```
SELECT   city,
         AVG(Salary) AS AverageSalary
FROM Employees
WHERE AVG(salary) > 65000;
GROUP BY city;
```

City	AVerageSalary
New York	75000
Toronto	75000
London	72000

Count the number of employees per department, including only departments with more than one employee

```
SELECT   department,
         COUNT(Employee ID) AS EmployeeCount
FROM employees
WHERE COUNT(Employee ID) > 1;
```

Department	Employee Count
IT	2
Finance	2

Retrieve the total price project budget per employee but for employees who are managing projects worth more than 150 000

```
SELECT e.EmployeeID
       FirstName
       LastName
       SUM(Budget) AS TotalProjectBudget,
FROM employees AS e
INNER JOIN projects AS p
ON e.EmployeeID = p.EmployeeID
WHERE SUM(Budget) > 150 000
GROUP BY Employee ID
```

EmployeeID	FirstName	LastName	TotalProjectBudget
1	John	Doe	175 000